

Assignment №2 - Filtering & Edge detection&image matching

Evaluation of this homework:

Blurring: a low pass filter	15%
Edge detection	15%
Harris Corner Detector and matching	35%
Report and code	35%

THIS ASSIGNMENT WILL BE 10% OF THE FINAL GRADE

Check your environment!

Be sure that you are using a working environment.

If you have any problems, find instructions in Moodle for setting up the environment.

Preparing images for low and high-pass filters

- Select one of the images you created in the previous assignment and apply the flowing steps on the image.
- Explain the important parameters for creating image filters and edge detection.
- Apply Blurring on your image by using a low pass filter Gaussian kernel, changing the kernel size and its effect on the image. Try with `kernel_ver = np.array([[0,1,-1,0], [1,3,-3,-1],[1,3,-3,-1],[0,1,-1,0]], np.float32)`
- Apply median and mean low pass filters.
- Compare different low pass filters with each other.
- Apply Canny edge-detection algorithm: Apply at least three different values of thresholds and compare results.

Important issues:

1. `Cv2.imwrite` expects uint8 data format as an input

<code>color_img</code>	Array of uint8	(300, 225, 3)	<code>[[[216 216 216] [216 216 216] [0.84705883 0.84705883 0.84705883] [0.84705883 0.84705883 0.847058 ...</code>
<code>color_img1</code>	Array of float32	(300, 225, 3)	

In this case, `color_img` will be saved correctly.

To save `color_img1` you need to convert float32 to uint8

2. Filters can be applied to both grayscale and colored images. However, using filters to colored images requires processing each color channel (RGB) separately.

Morphological operations: A set of operations that process images according to their shapes.

- Use text images to apply Morphological operations of Erosion and Dilation.
- Use text images to apply morphological operations: opening, closing, and morphological gradient.
- Define kernel and try to choose several options: 3, 5, or 7.

Important issues:

1. Low-pass filters can be applied to both grayscale and colored images. However, using filters to colored images requires processing each color channel (RGB) separately.

Harris Corner Detector and Feature Matching

- Choose two images with slight shifts between them (building from close points of view).

- **Apply the Harris Corner Detector:** Ensure the input image is in grayscale and of float32 type. Perform this operation once by manually applying all steps and a second time by using the function `cv2.cornerHarris`. What are the differences between the outputs?
- **Application of the Shi-Tomasi Corner Detector:** Apply the Shi-Tomasi Corner Detector to the same image and test various corner selection criteria. Examine how different parameters influence the detection outcome.
- **Use of FAST and ORB for Feature Matching:** Utilize two images containing similar information but captured from different viewpoints. The task involves using FAST for corner detection and ORB for feature matching across these images. Apply the same procedure with the Features from the Accelerated Segment Test (FAST) and the ORB (Oriented FAST and Rotated BRIEF) detectors and compare the matching results to one of the models discussed in the lessons (LIFT, for example). Incorporate the outcomes of filtering matches.
- **Explanation of Algorithm Differences:** Explain the differences in algorithms between the various models.

Important issues:

1. You should implement three approaches: FAST, ORB, and one of your choice (for example, LIFT). Then compare the results and describe how each algorithm succeed in the matching.

[What to submit?](#)

PDF with:

1. Explain the important parameters that need to be considered in creating image filters and edge detection.
2. Include images showing the application of low-pass filtering and high-pass filtering with various kernel sizes and threshold values. (2 examples for each will be enough)
3. Include images showing the application of edge detection.
4. Describe the differences between implementing different kernel sizes in edge detection. How do you choose the optimal size?
5. Include images showing the application of corner detection.
6. Compare different algorithms between each other.
7. Include images of feature matching.

Your code:

1. Submit your script as a single .py file (or .jpynb)

IMPORTANT!!!

I am not a Hebrew speaker, please submit your assignments in English

1. Add all images, tables and text to a **single** .docx or .pdf (please keep reasonable resolution of images)
2. I expect to receive only **two files** from each of you: **report** (.pdf or .doc or .docx) and **code** (.py or .jpynb)
3. Add your name to the name of the report and code

Example of naming:

OK • laroslav_Groz dov_assignment_2.pdf

OK • laroslav_Groz dov_assignment_2.py

NOT OK • assignment_2_ laroslav_Groz dov.pdf

NOT OK • Code_ laroslav_Groz dov.py

NOT OK • %anything_else_than_your_name%_assignment_2.pdf

DON'T SEND YOUR IMAGES AND VIDEOS!

Moodle have limit of 50mb for uploads so to avoid it I prefer to receive your assignments on Google Drive.

Link for submission:

<https://forms.gle/34hCxW8c7KNtjNpG7>